

Credit Card Approval Prediction

(by Using Machine Learning)

Project Description:

The Credit Card Approval Predictor is an intelligent, AI-driven web platform designed to streamline the credit evaluation process. By assessing an applicant's financial background and personal details, the system accurately forecasts their eligibility for a new credit line. Throughout the development phase, various machine learning models—including Logistic Regression, Decision Trees, Random Forest, and XGBoost—were trained and evaluated on historical data. The top-performing model was then integrated into a Flask backend, allowing the application to deliver instant, real-time credit decisions through a clean, interactive user interface. Ultimately, this automated approach significantly reduces manual workload, accelerates processing times, and ensures unbiased outcomes for financial organizations.

Problem Statement

Traditional, manual credit evaluation workflows are frequently time-consuming, subjective, and vulnerable to human error. This inefficiency can result in the unfair rejection of creditworthy individuals or the risky approval of unqualified applicants. To overcome these challenges, there is a critical need for a robust, automated solution that leverages machine learning to instantly classify incoming applications. Implementing such a data-driven system enables financial institutions to drastically enhance their operational efficiency, decision consistency, and overall reliability.

Scenarios:

Scenario 1: Standard Credit Card Application

A customer applies for a credit card by entering their employment details, annual income, education level, housing status, and family information through the web application. The system preprocesses the input data and uses the trained Machine Learning model to predict whether the application should be **Approved** or **Rejected**, displaying the result instantly.

Scenario 2: High-Risk Applicant Evaluation

A bank officer evaluates an applicant with an unstable employment history and relatively low annual income. After submitting the applicant's details, the system analyses multiple financial and demographic factors using the trained model and predicts a **Rejected** status, helping the bank minimize potential financial risk and make informed lending decisions.

Scenario 3: Bulk Customer Screening

A financial institution processes numerous credit card applications daily. Bank employees enter each applicant's information into the system, which rapidly evaluates every application using the deployed Machine Learning model. The application provides immediate approval

predictions, significantly reducing manual verification time, improving operational efficiency, and maintaining consistent decision-making across all applications.

Objectives

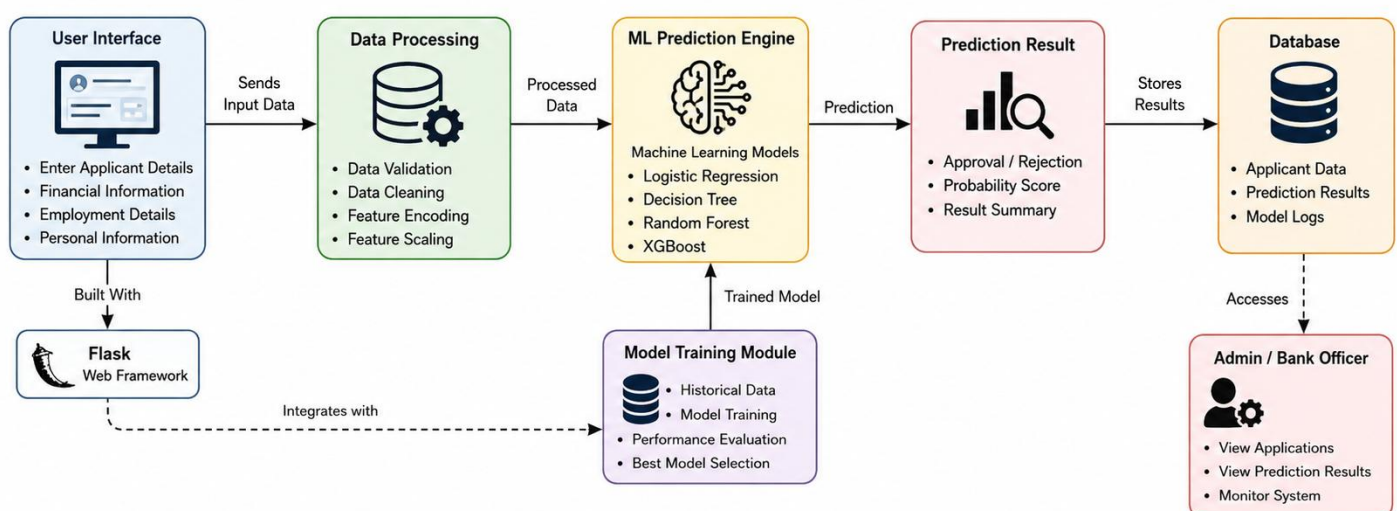
- To analyse and preprocess a real-world credit card application dataset (clean_dataset.csv) for use in machine learning models.
- To engineer meaningful features, such as the Debt-to-Income Ratio, that improve predictive performance.
- To train and compare multiple classification algorithms — Logistic Regression, Decision Tree, Random Forest, and XGBoost — for predicting credit card approval outcomes.
- To evaluate model performance using standard classification metrics including Accuracy, Precision, Recall, F1-Score, Confusion Matrix, and ROC-AUC.
- To deploy the best-performing model as an accessible web application for real-time predictions.

Scope

The scope of this project is limited to binary classification of credit card applications (Approved / Rejected) using the provided dataset. The system is intended as an academic demonstration of an end-to-end machine learning workflow — from data preprocessing to deployment — and is not intended for production use in an actual financial institution without further regulatory, fairness, and security review.

Technical Architecture:

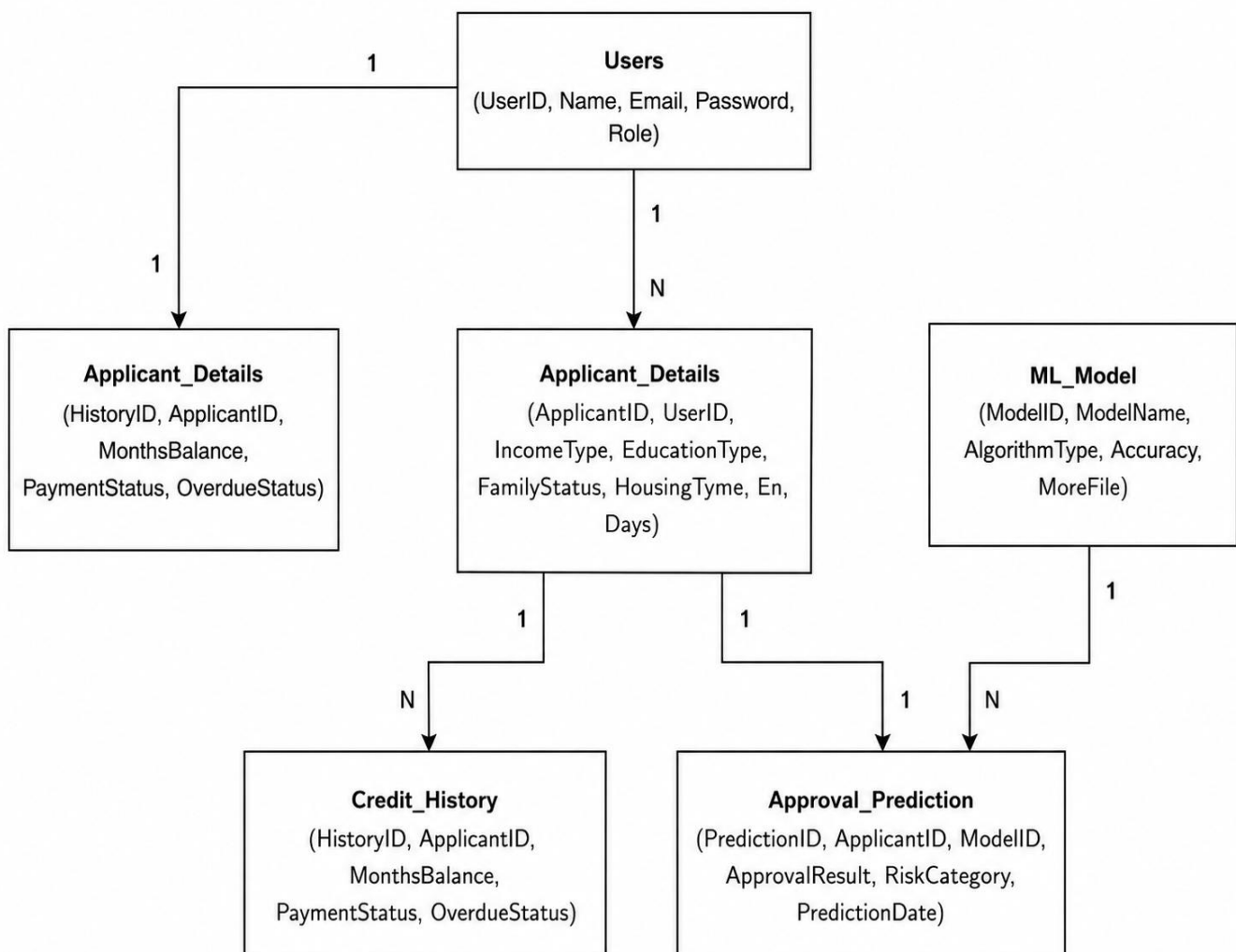
Architecture Flow



ER Diagram

Description:

The **Credit Card Approval Prediction System** follows a modular machine learning architecture to provide fast and accurate credit card approval decisions. Users enter applicant information through a **Flask-based web application**, where the input data is validated, cleaned, encoded, and pre-processed before being sent to the **Machine Learning Prediction Engine**. The system evaluates the application using trained models such as **Logistic Regression, Decision Tree, Random Forest, and XGBoost**, with the best-performing model selected for real-time predictions. The prediction result (Approved or Rejected) is displayed instantly and stored for future reference and analysis. This architecture enables financial institutions to automate the credit card approval process, improve decision-making accuracy, reduce manual effort, and deliver a seamless user experience.



Pre-requisites

- Python Programming Proficiency (Python 3.x)
- Machine Learning Fundamentals — scikit-learn, XGBoost
- Data Handling Libraries — Pandas, NumPy
- Data Visualization — Matplotlib / Seaborn (for EDA)
- Web Framework Knowledge — Flask, for deployment of the prediction interface
- Version Control with Git

Project Workflow:

Milestone 1: Data Collection and Understanding :

- Activity 1.1: Import the dataset (clean_dataset.csv) and inspect its structure, columns, and data types.
- Activity 1.2: Perform exploratory data analysis (EDA) to understand feature distributions and class balance between Approved and Rejected applications.
- Activity 1.3: Identify categorical and numerical features relevant to the prediction task.

Milestone 2: Data Preprocessing and Feature Engineering

- Activity 2.1: Verify the dataset for missing values and handle them appropriately.
- Activity 2.2: Check for and remove duplicate records to ensure data quality.
- Activity 2.3: Apply one-hot encoding to categorical variables to convert them into a numerical format suitable for machine learning models.
- Activity 2.4: Apply a log transformation to the Income feature to reduce skewness and stabilize variance.
- Activity 2.5: Engineer a new Debt-to-Income Ratio feature to better capture applicant financial risk.
- Activity 2.6: Split the dataset into training and testing sets.
- Activity 2.7: Apply StandardScaler to numerical features where appropriate, to normalize feature ranges for scale-sensitive algorithms.

Milestone 3: Model Building

- Activity 3.1: Train a Logistic Regression model as a baseline classifier.
- Activity 3.2: Train a Decision Tree classifier to capture non-linear decision boundaries.
- Activity 3.3: Train a Random Forest ensemble model to improve generalization and reduce overfitting.
- Activity 3.4: Train an XGBoost model to leverage gradient boosting for potentially higher predictive accuracy.

Milestone 4: Model Evaluation and Selection

- Activity 4.1: Evaluate each trained model on the test set using Accuracy, Precision, Recall, and F1-Score.
- Activity 4.2: Generate a Confusion Matrix for each model to analyse true/false positive and negative predictions.
- Activity 4.3: Compute the ROC-AUC score to assess the models' ability to discriminate between classes.
- Activity 4.4: Compare all models and select the best-performing algorithm for deployment. Based on the evaluation results, the Random Forest Classifier achieved a high accuracy of 88.41% with an exceptional ROC-AUC score of 95.18%. Due to its robustness against overfitting and high precision (90.91%), Random Forest was selected as the final model for deployment.

Milestone 5: Deployment

- Activity 5.1: Save the best-performing trained model (Random Forest) and the categorical LabelEncoders using joblib for reuse in the web application.
- Activity 5.2: Build a Flask-based web application with an HTML/CSS front-end that collects applicant details through an application form.
- Activity 5.3: Integrate the trained model with the Flask backend (app.py) to generate real time approval predictions via a /predict API endpoint.
- Activity 5.4: Deploy the application on Render, making it publicly accessible as a live web service.
- Activity 5.5: Test the live application end-to-end to verify correct functioning, from form submission to prediction display.

The application has been successfully deployed and is live at the following URL:

<https://ai-ml-smartbridge-project-credit-card.onrender.com>

Full implementation details, source code, and screenshots of the deployed web application are provided in Section 6 (Web Application).

Methodology

Dataset Description

Attribute	Details
Dataset Name	clean_dataset.csv
Type	Structed
Target Variable	Credit Card Approval Status(accepted/rejected)
Feature Types	Categorical & Numerical attributes of attributes

Data Preprocessing Steps

- **Data Loading and Inspection:** The dataset (690 records, 16 columns) was loaded using Pandas, followed by inspection using `df.head()`, `df.tail()`, `df.info()`, and `df.describe()` to understand structure, data types, and summary statistics.
- **Missing Value Verification:** The dataset was checked for missing/null values prior to model training to ensure data completeness.
- **Duplicate Checking:** Duplicate records were identified and handled to prevent data leakage and bias.
- **Categorical Encoding:** Categorical features (Industry, Ethnicity, Citizen) were encoded using Label Encoding / One-Hot Encoding to convert them into a numerical format suitable for machine learning models.
- **Log Transformation:** The Income feature was log-transformed to reduce right-skewness caused by extreme high-income values (up to ₹100,000).
- **Feature Engineering:** A Debt-to-Income Ratio feature was derived to capture applicant financial risk more effectively.
- **Train-Test Split:** The dataset was divided into training (80%) and testing (20%) subsets using `train_test_split` from scikit-learn to evaluate model generalization.
- **Feature Scaling:** `StandardScaler` was applied to numerical features where required by the algorithm (e.g., Logistic Regression).

Algorithms Used

-  Logistic Regression
-  Decision Tree Captures
-  Random Forest
-  XGBoost

Evaluation Metrics

- **Accuracy** — Overall proportion of correctly classified applications.
- **Precision** — Proportion of predicted approvals that were actually correct.
- **Recall** — Proportion of actual approvals that were correctly identified.
- **F1-Score** — Harmonic mean of Precision and Recall, useful for imbalanced classes.
- **Confusion Matrix** — Detailed breakdown of true/false positives and negatives
- **ROC-AUC** — Measures the model's ability to distinguish between approved and rejected classes across thresholds

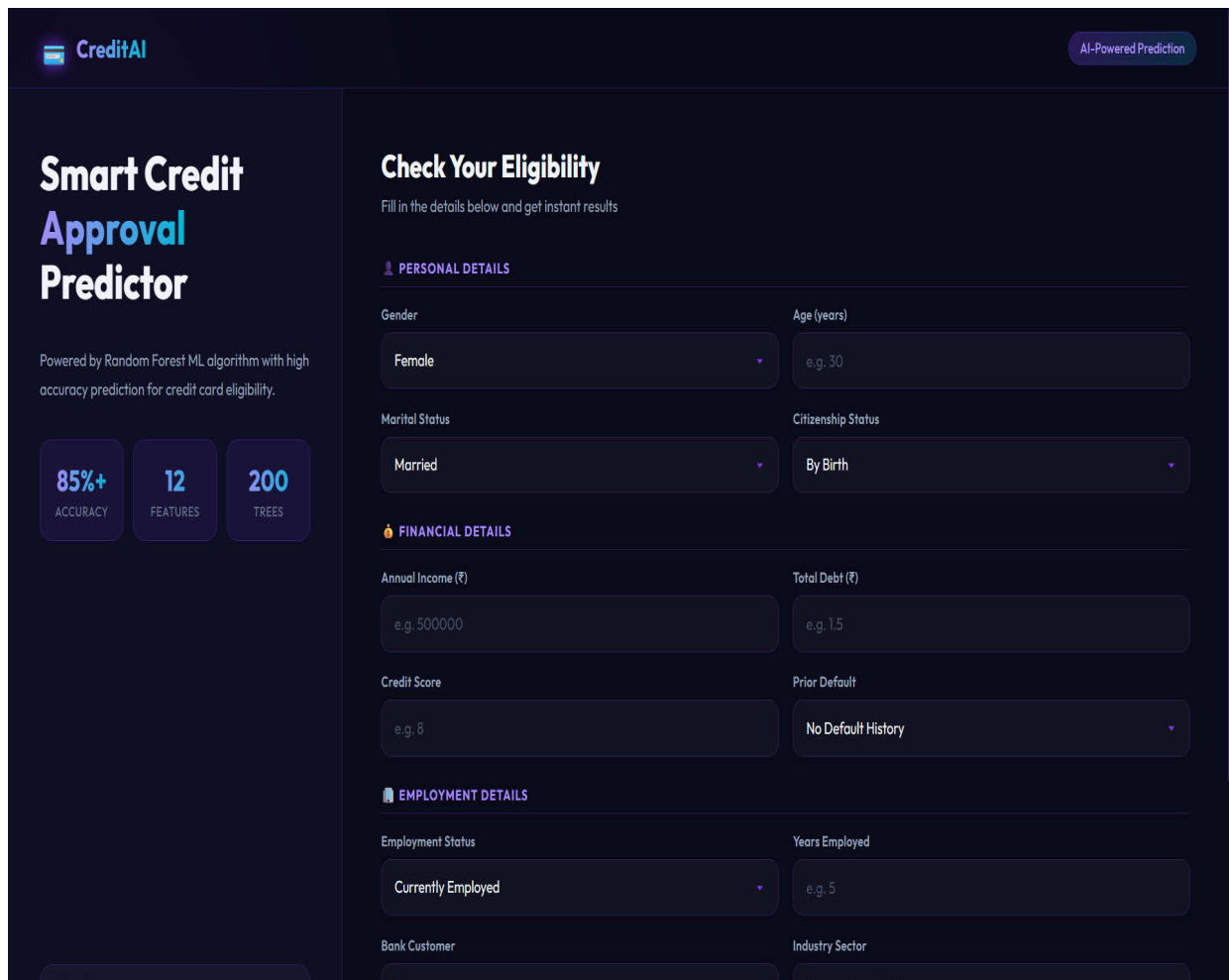
Web Application

The trained XGBoost model has been deployed as an interactive web application named "Credit Card Approval Prediction", built using Flask (backend) and HTML/CSS/JavaScript (frontend). The application allows a user to enter applicant details through a form and receive an instant Approved / Not Approved prediction along with a confidence percentage.

The application has been successfully deployed and is publicly accessible at:

<https://ai-ml-credit-card-approval-prediction-1.onrender.com/>

Application Form



The screenshot displays the "Smart Credit Approval Predictor" web application. The interface is dark-themed with a sidebar on the left and a main content area on the right. The sidebar features the "CreditAI" logo, the title "Smart Credit Approval Predictor", a description "Powered by Random Forest ML algorithm with high accuracy prediction for credit card eligibility.", and three statistics: "85%+ ACCURACY", "12 FEATURES", and "200 TREES". The main content area is titled "Check Your Eligibility" and includes a sub-header "Fill in the details below and get instant results". The form is divided into four sections: "PERSONAL DETAILS" (Gender: Female, Age: 30), "Marital Status" (Married), "Citizenship Status" (By Birth), "FINANCIAL DETAILS" (Annual Income: 500,000, Total Debt: 1.5, Credit Score: 8, Prior Default: No Default History), and "EMPLOYMENT DETAILS" (Employment Status: Currently Employed, Years Employed: 5, Bank Customer, Industry Sector).

 CreditAI

AI-Powered Prediction

Smart Credit Approval Predictor

Powered by Random Forest ML algorithm with high accuracy prediction for credit card eligibility.

85%+
ACCURACY

12
FEATURES

200
TREES

Check Your Eligibility

Fill in the details below and get instant results

PERSONAL DETAILS

Gender

Female

Age (years)

e.g. 30

Marital Status

Married

Citizenship Status

By Birth

FINANCIAL DETAILS

Annual Income (₹)

e.g. 500000

Total Debt (₹)

e.g. 1.5

Credit Score

e.g. 8

Prior Default

No Default History

EMPLOYMENT DETAILS

Employment Status

Currently Employed

Years Employed

e.g. 5

Bank Customer

Industry Sector

Prediction of Result

 **CreditAI**

20000

15000

Credit Score

4

Prior Default

No Default History

 **EMPLOYMENT DETAILS**

Employment Status

Not Employed

Years Employed

2

Bank Customer

No

Industry Sector

Information Technology

 **Predict Eligibility**


APPROVED

Model Confidence: 88.5%

Your application meets the eligibility criteria. You are likely to get approved!

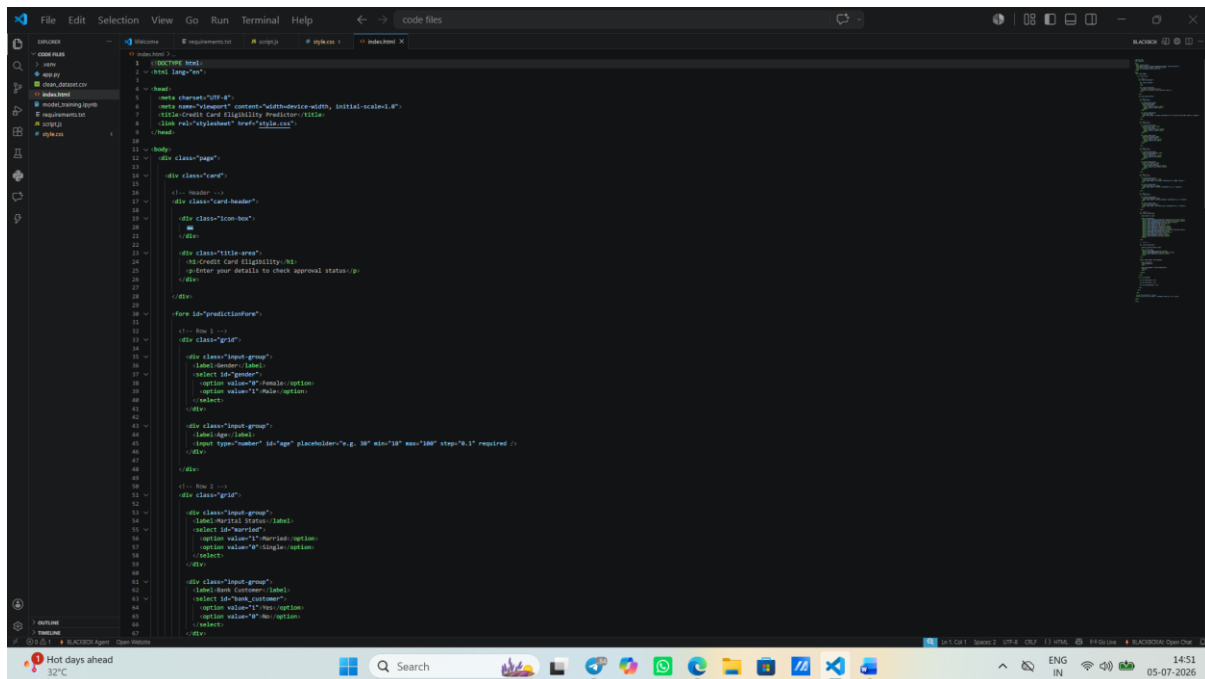
KN

Kadali Neeraj
23B91A6167 - B.Tech CSE (AI & ML)
Sagi Ramakrishnam Raju Engineering
College
2023 - 2027 - Smartbridge AICTE Internship

Frontend Implementation

- HTML
- CSS
- JavaScript

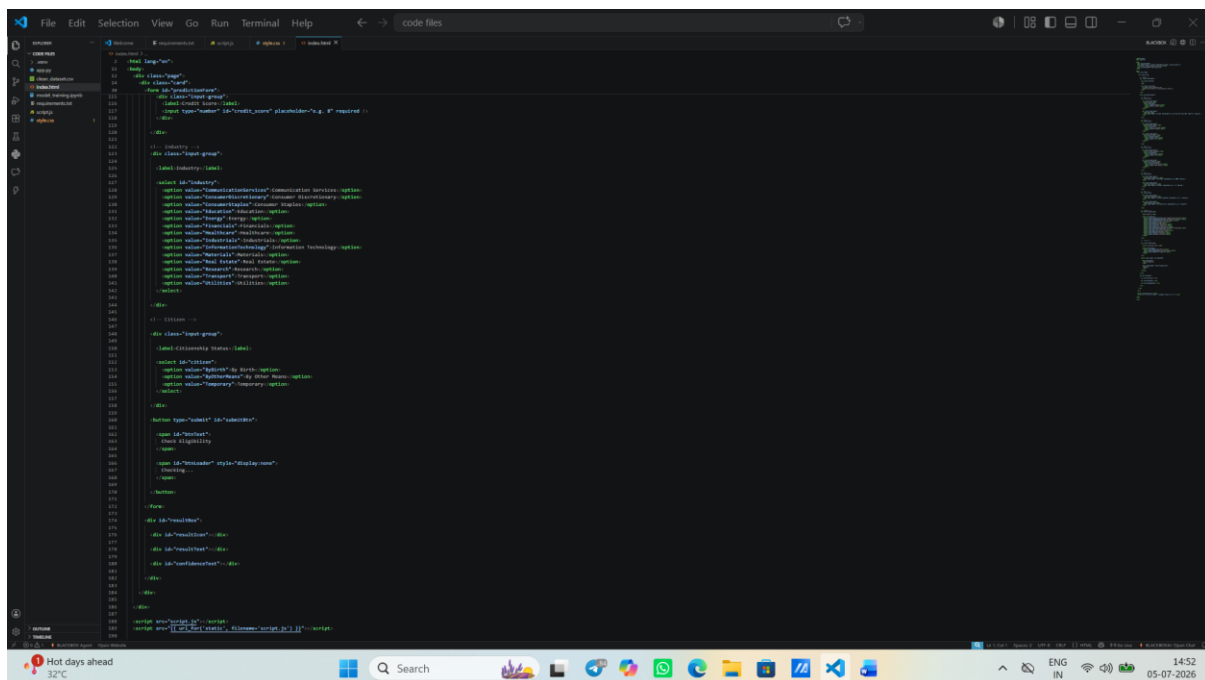
HTML(structure)



```

1 <!-- Credit Card Eligibility No -->
2 <html lang="en">
3
4 <head>
5   <meta charset="UTF-8">
6   <meta name="viewport" content="width=device-width, initial-scale=1.0">
7   <title>Credit Card Eligibility Prediction - HTML</title>
8   <link rel="stylesheet" href="style.css">
9 </head>
10
11 <body>
12   <div class="page">
13     <div class="card">
14       <!-- Header -->
15       <div class="card-header">
16         <div class="icon-box">
17           <img alt="Credit Card Icon" data-bbox="200 310 230 340"/>
18         </div>
19         <div class="title-area">
20           <h3>Credit Card Eligibility No</h3>
21           <p>Enter your details to check approval status</p>
22         </div>
23       </div>
24       <form id="predictionForm">
25         <!-- Step 1 -->
26         <div class="grid">
27           <div class="input-group">
28             <label>Gender</label>
29             <select id="gender">
30               <option value="F">Female</option>
31               <option value="M">Male</option>
32             </select>
33           </div>
34           <div class="input-group">
35             <label>Age</label>
36             <input type="number" id="age" placeholder="e.g. 30" min="18" max="100" step="0.1" required />
37           </div>
38         </div>
39         <!-- Step 2 -->
40         <div class="grid">
41           <div class="input-group">
42             <label>Marital Status</label>
43             <select id="married">
44               <option value="1">Married</option>
45               <option value="0">Single</option>
46             </select>
47           </div>
48           <div class="input-group">
49             <label>Bank Customer</label>
50             <select id="bank_customer">
51               <option value="1">Yes</option>
52               <option value="0">No</option>
53             </select>
54           </div>
55         </div>
56       </form>
57     </div>
58   </div>
59 </body>
60 </html>

```

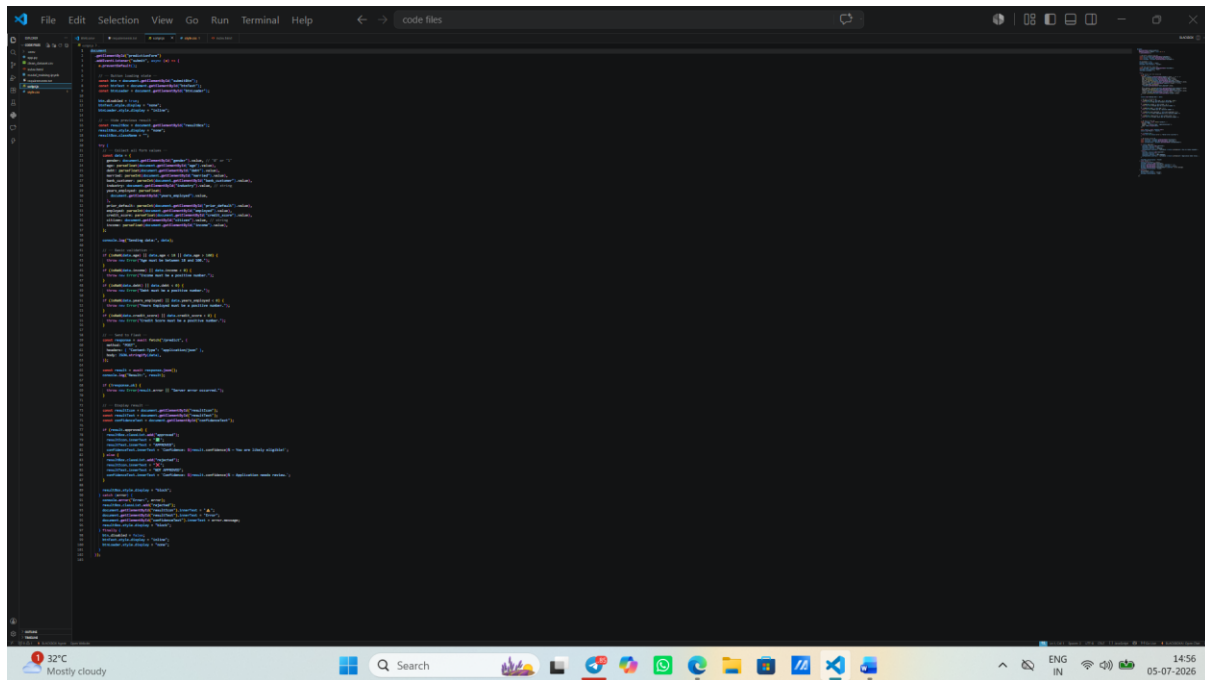


```

101 <!-- Credit Card Eligibility No -->
102 <html lang="en">
103
104 <head>
105   <meta charset="UTF-8">
106   <meta name="viewport" content="width=device-width, initial-scale=1.0">
107   <title>Credit Card Eligibility Prediction - HTML</title>
108   <link rel="stylesheet" href="style.css">
109 </head>
110
111 <body>
112   <div class="page">
113     <div class="card">
114       <!-- Header -->
115       <div class="card-header">
116         <div class="icon-box">
117           <img alt="Credit Card Icon" data-bbox="200 310 230 340"/>
118         </div>
119         <div class="title-area">
120           <h3>Credit Card Eligibility No</h3>
121           <p>Enter your details to check approval status</p>
122         </div>
123       </div>
124       <form id="predictionForm">
125         <!-- Step 1 -->
126         <div class="grid">
127           <div class="input-group">
128             <label>Gender</label>
129             <select id="gender">
130               <option value="F">Female</option>
131               <option value="M">Male</option>
132             </select>
133           </div>
134           <div class="input-group">
135             <label>Age</label>
136             <input type="number" id="age" placeholder="e.g. 30" min="18" max="100" step="0.1" required />
137           </div>
138         </div>
139         <!-- Step 2 -->
140         <div class="grid">
141           <div class="input-group">
142             <label>Marital Status</label>
143             <select id="married">
144               <option value="1">Married</option>
145               <option value="0">Single</option>
146             </select>
147           </div>
148           <div class="input-group">
149             <label>Bank Customer</label>
150             <select id="bank_customer">
151               <option value="1">Yes</option>
152               <option value="0">No</option>
153             </select>
154           </div>
155         </div>
156       </form>
157     </div>
158   </div>
159 </body>
160 </html>

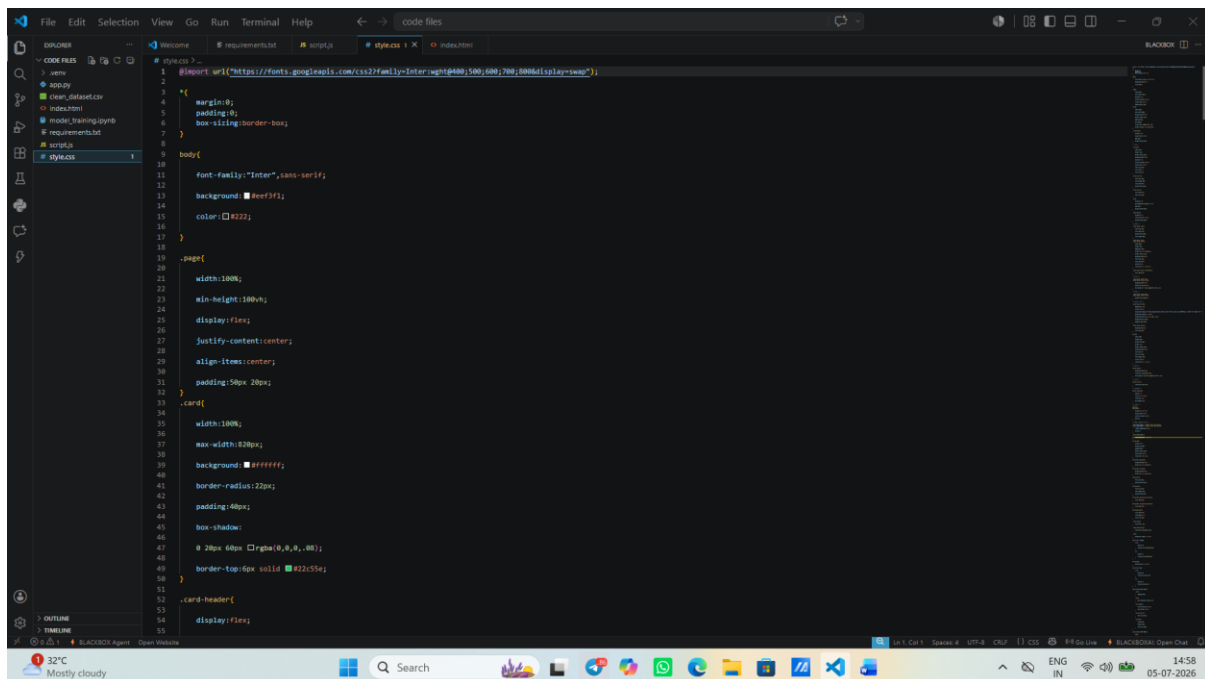
```

JavaScript



The screenshot shows a code editor with a dark theme. The file explorer on the left shows a project structure with files like 'index.html', 'script.js', and 'style.css'. The main editor area displays JavaScript code for a web application. The code includes a 'DOMContentLoaded' event listener that triggers a function to load data from a JSON file. The function uses 'fetch' to retrieve data and 'JSON.parse' to convert it to a JavaScript object. The data is then used to populate a table with rows and columns. The table has a header row with columns for 'Name', 'Age', 'Gender', 'Address', and 'Phone'. The data rows are generated from the JSON array. The code also includes a 'loadData' function that is called when the 'Load Data' button is clicked. The 'loadData' function calls the 'loadDataFromJSON' function and updates the table with the new data. The code is well-commented and uses modern JavaScript syntax.

CSS



The screenshot shows a code editor with a dark theme. The file explorer on the left shows a project structure with files like 'index.html', 'script.js', and 'style.css'. The main editor area displays CSS code for a web application. The code defines styles for the body, .page, .card, and .card-header. The body has a font-family of 'Inter', sans-serif, a background color of #f0f0f0, and a color of #333. The .page has a width of 100%, a min-height of 100vh, and a display of flex. The .card has a width of 100%, a max-width of 800px, a background color of #ffffff, a border-radius of 20px, a padding of 40px, a box-shadow, and a border-top of 4px solid #22c55e. The .card-header has a display of flex. The code is well-commented and uses modern CSS syntax.

Conclusion and Future Scope

Conclusion

The Credit Card Approval Prediction using Machine Learning project successfully demonstrates how machine learning can be applied to automate and improve the credit card approval process. By analysing applicant information such as income, employment status, education, family status, housing type, and financial history, the developed model can accurately predict whether a credit card application is likely to be approved or rejected. Various classification algorithms were evaluated, and the best-performing model was integrated into a Flask-based web application to provide real-time predictions through a user-friendly interface. This system helps reduce manual effort, minimizes human bias, speeds up decision-making, and supports financial institutions in making consistent and reliable credit approval decisions. Overall, the project proves that machine learning is an effective solution for enhancing the efficiency, accuracy, and transparency of the credit card approval process.

Future Scope

The proposed Credit Card Approval Prediction system can be further enhanced with several advanced features to improve its performance and real-world usability. Future improvements include:

- Integrating larger and more diverse datasets to improve model accuracy and generalization.
- Deploying the application on cloud platforms such as AWS, Azure, or Google Cloud for better scalability and accessibility.
- Implementing deep learning techniques and ensemble learning methods to further enhance prediction performance.
- Adding Explainable AI (XAI) techniques such as SHAP or LIME to provide transparent explanations for each approval decision.
- Integrating real-time credit score and banking APIs to make predictions using live financial data.
- Incorporating fraud detection mechanisms to identify suspicious or fraudulent credit card applications.
- Developing a mobile application for easier access by customers and bank officials.
- Continuously retraining the model with new application data to adapt to changing customer behavior and financial trends.
- Implementing role-based authentication and enhanced security measures to protect sensitive customer information.
- Extending the system to support loan approval, insurance eligibility, and other financial risk assessment applications using the same machine learning framework.